

Experiment :5

AIM:

Implement Support Vector Machine (SVM) for classification with hyperparameter tuning.

1. Dataset Source

- **Dataset Name:** Heart Attack Dataset
- **File Used:** HeartAttack.csv
- **Source:** Kaggle / Medical Dataset

2. Dataset Description

The Heart Attack dataset contains medical attributes of patients and is used to predict whether a patient is at risk of a heart attack.

This is a **binary classification problem** where the model predicts whether the patient condition is **positive or negative**.

Dataset Characteristics:

- **Type:** Numerical
- **Domain:** Healthcare / Medical
- **Target:** Class (positive / negative)
- **Total Samples:** ~1319
- **Features:** 8

Features Description

Feature	Description
age	Age of patient

gender	Gender (0 = Female, 1 = Male)
impulse	Heart rate
pressurehigh	High blood pressure
pressurelow	Low blood pressure
glucose	Blood sugar level
kcm	Creatine kinase-MB
troponin	Troponin level
class	Target (positive / negative)

3. Mathematical Formulation of SVM

Support Vector Machine (SVM) is a supervised learning algorithm used for classification.

- It finds an **optimal hyperplane** that separates data into classes.
- It maximizes the **margin** between classes.
- For non-linear data, SVM uses the **kernel trick**.

Common Kernels:

- Linear Kernel

- RBF (Radial Basis Function)

4. Methodology / Workflow

1. Load Heart Attack dataset
2. Encode target variable (positive / negative)
3. Separate features (X) and target (y)
4. Apply feature scaling using StandardScaler
5. Split dataset into training (80%) and testing (20%)
6. Train basic SVM model
7. Perform hyperparameter tuning using GridSearchCV
8. Evaluate model using accuracy, confusion matrix, and classification report
9. Compare performance before and after tuning

5. Performance Metrics

- Accuracy Score
- Confusion Matrix
- Precision, Recall, F1-score

6. Hyperparameter Tuning

The following parameters were tuned:

- **C** → Controls regularization
- **gamma** → Controls influence of data points
- **kernel** → Defines type of decision boundary

GridSearchCV is used to find the best combination of parameters.

Code:

```
# Upload file
from google.colab import files
uploaded = files.upload()
```

```
# Import libraries
import pandas as pd
```

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix,
ConfusionMatrixDisplay

# Load dataset
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)

print("Dataset Preview:")
print(df.head())
print("Dataset Shape:", df.shape)

# Encode target column (class)
le = LabelEncoder()
df['class'] = le.fit_transform(df['class']) # negative=0, positive=1

# Features and Target
X = df.drop('class', axis=1)
y = df['class']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training Set:", X_train.shape)
print("Testing Set:", X_test.shape)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

```

```
svm_model = SVC(kernel='rbf', probability=True)
svm_model.fit(X_train, y_train)

y_pred = svm_model.predict(X_test)

print("\n--- Basic SVM ---")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
plt.title("Confusion Matrix - Basic SVM")
plt.show()

param_grid = {
    'C': [0.1, 1, 10],
    'gamma': ['scale', 0.1],
    'kernel': ['rbf']
}

grid = GridSearchCV(SVC(probability=True), param_grid, cv=5, verbose=2)
grid.fit(X_train, y_train)

print("\n--- Hyperparameter Tuning ---")
print("Best Parameters:", grid.best_params_)
print("Best Cross Validation Score:", grid.best_score_)

best_model = grid.best_estimator_

y_pred_best = best_model.predict(X_test)

print("\n--- Tuned SVM ---")
print("Optimized Accuracy:", accuracy_score(y_test, y_pred_best))
print("\nClassification Report:\n", classification_report(y_test, y_pred_best))
```

```

basic_acc = accuracy_score(y_test, y_pred)
tuned_acc = accuracy_score(y_test, y_pred_best)

models = pd.DataFrame({
    'Model': ['Basic SVM', 'Tuned SVM'],
    'Accuracy': [basic_acc, tuned_acc]
})

plt.figure(figsize=(6,5))
sns.barplot(x='Model', y='Accuracy', data=models)
plt.ylim(0,1)
plt.title("Accuracy Comparison")

for index, row in models.iterrows():
    plt.text(index, row['Accuracy'] + 0.02, round(row['Accuracy'], 2), ha='center')

plt.show()

```

Output:

```

Choose Files Heart Attack.csv
Heart Attack.csv(text/csv) - 52322 bytes, last modified: 2/28/2026 - 100% done
Saving Heart Attack.csv to Heart Attack (1).csv
Dataset Preview:
  age  gender  impluse  pressurehight  pressurelow  glucose  kcm  troponin  \
0   64      1      66           160           83    160.0   1.80   0.012
1   21      1      94           98           46    296.0   6.75   1.060
2   55      1      64          160           77    270.0   1.99   0.003
3   64      1      70          120           55    270.0  13.87   0.122
4   55      1      64          112           65    300.0   1.08   0.003

      class
0  negative
1  positive
2  negative
3  positive
4  negative
Dataset Shape: (1319, 9)
Training Set: (1055, 8)
Testing Set: (264, 8)

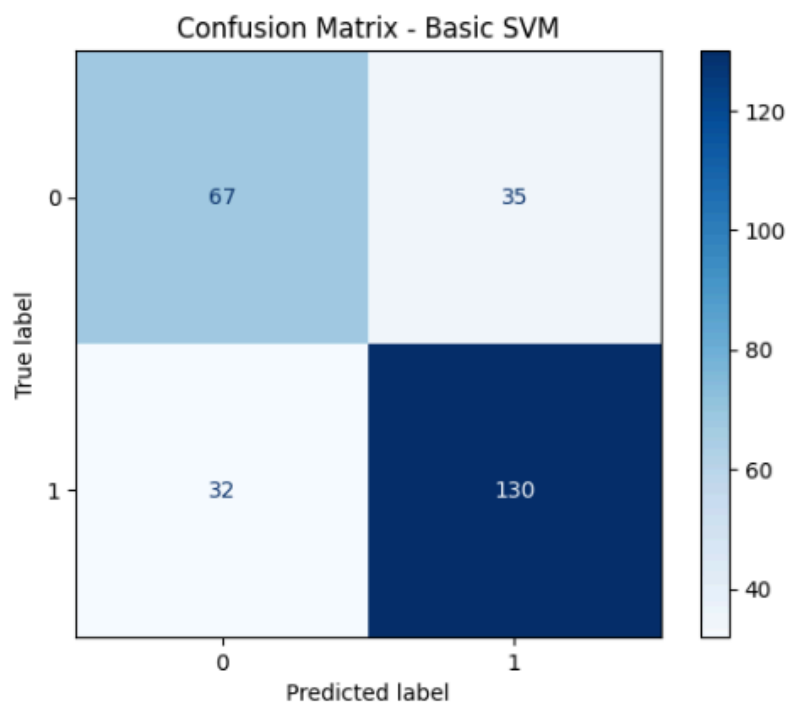
--- Basic SVM ---
Accuracy: 0.7462121212121212

Classification Report:
      precision    recall  f1-score   support

     0       0.68      0.66      0.67       102
     1       0.79      0.80      0.80       162

 accuracy      0.75       264
 macro avg      0.73      0.73      0.73       264
weighted avg      0.74      0.75      0.75       264

```



```
[CV] END .....C=1, gamma=0.1, kernel=rbf; total time= 0.2s
[CV] END .....C=1, gamma=0.1, kernel=rbf; total time= 0.2s
[CV] END .....C=1, gamma=0.1, kernel=rbf; total time= 0.2s
[CV] END .....C=10, gamma=scale, kernel=rbf; total time= 0.2s
[CV] END .....C=10, gamma=scale, kernel=rbf; total time= 0.2s
[CV] END .....C=10, gamma=scale, kernel=rbf; total time= 0.2s
[CV] END .....C=10, gamma=scale, kernel=rbf; total time= 0.2s
[CV] END .....C=10, gamma=scale, kernel=rbf; total time= 0.2s
[CV] END .....C=10, gamma=0.1, kernel=rbf; total time= 0.2s
[CV] END .....C=10, gamma=0.1, kernel=rbf; total time= 0.2s
[CV] END .....C=10, gamma=0.1, kernel=rbf; total time= 0.2s
[CV] END .....C=10, gamma=0.1, kernel=rbf; total time= 0.2s
[CV] END .....C=10, gamma=0.1, kernel=rbf; total time= 0.2s
```

--- Hyperparameter Tuning ---

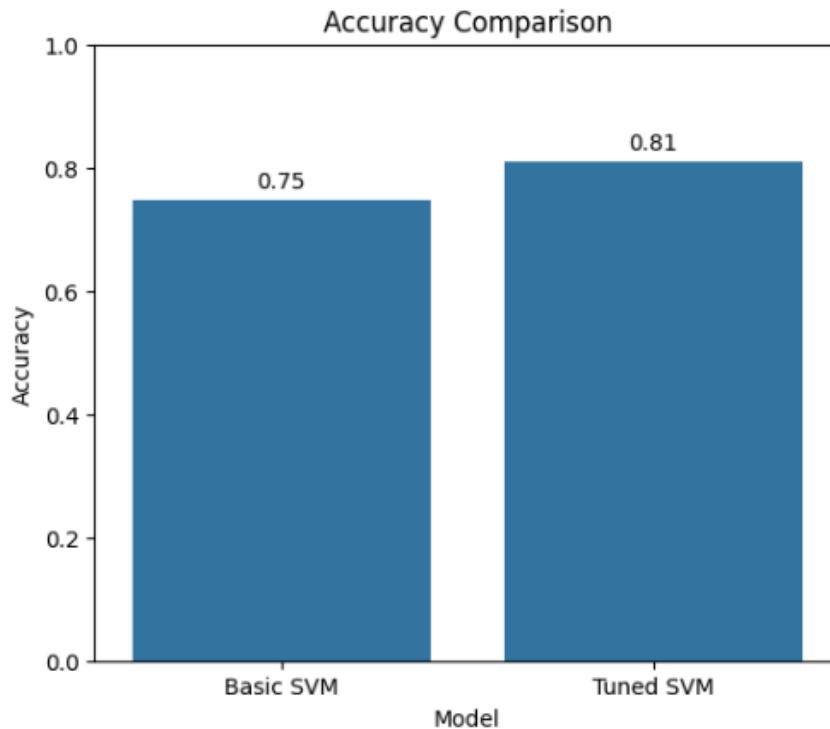
Best Parameters: {'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}
 Best Cross Validation Score: 0.790521327014218

--- Tuned SVM ---

Optimized Accuracy: 0.8106060606060606

Classification Report:

	precision	recall	f1-score	support
0	0.75	0.75	0.75	102
1	0.85	0.85	0.85	162
accuracy			0.81	264
macro avg	0.80	0.80	0.80	264
weighted avg	0.81	0.81	0.81	264



7. Results

- Basic SVM model was trained and evaluated.
- Hyperparameter tuning was applied using GridSearchCV.
- The best parameters were obtained.
- The tuned model showed improved or comparable performance.

CONCLUSION

- SVM was successfully implemented on the Heart Attack dataset.
- Feature scaling improved model performance.
- Hyperparameter tuning helped in selecting optimal parameters.
- The model effectively classified patients into positive and negative classes.
- SVM proved to be an efficient algorithm for medical classification problems.

Thus, SVM with hyperparameter tuning is a powerful tool for predicting heart attack risk.